

Aujourd'hui

- Perfectionnement de la classe TableauInt
 - Opérateur []
 - Changement de taille
 - Copie de tableaux
 - Constructeur de copie

Rappel de la classe TableauInt

TableauInt
tableau_ : int * taille_ : int
TableauInt(taille:int) ~TableauInt() getTaille() : int getElement(i:int) : int setElement(i:int, val:int)

Exemple d'utilisation

```
void main()
{
    int n,i;
    cout<<"Quelle taille voulez-vous ?";
    cin>>n;
    TableauInt t(n);
    for (i=0;i<t.getTaille();i++)
        t.setElement(i,2*i);
    cout<<"t :";
    for (i=0;i<t.getTaille();i++)
        cout<<t.getElement(i)<<" ";
    cout<<endl;
}
```

Accès par l'opérateur []

```
void main()
{
    int n,i;
    cout<<"Quelle taille voulez-vous ?";
    cin>>n;
    TableauInt t(n);
    for (i=0;i<t.getTaille();i++)
        t[i]=2*i;
    cout<<"t :";
    for (i=0;i<t.getTaille();i++)
        cout<<t[i]<<" ";
    cout<<endl;
}
```

Méthode operator []

```
class TableauInt
{
    ... // Champs
public :
    ... // Méthodes
    int & operator[](int i);
};
```

```
int &TableauInt::operator[](int i)
{
    if (i<0 || i>=taille_)
    {
        cerr<<"Dépassement d'indice : i = "<<i;
        abort();
    }
    return tableau_[i];
}
```

Paramètre de retour transmis par référence

- Dans la déclaration : `int & operator [ ]( int i);`
 - *int &* indique que la valeur de retour est retournée par référence
 - C'est à dire que c'est la référence de l'élément *tableau_[i]* qui est retournée
 - Cet élément est donc utilisable comme une variable :
 - Il peut être affecté : `t[i] = 2*i;`
 - Il peut être lu : `cout<<t[i]<<" ";`

Changement de taille

- Ce que l'on veut faire
- Principe
- Mise en œuvre

Ce que l'on veut faire

- Changement de taille

```
TableauInt t(5);
```

```
int i;
```

```
for (i=0;i<5;i++) t[i]=1;
```

```
t.setTaille(10);
```

```
for (i=5;i<10;i++) t[i]=2;
```

```
for (i=0;i<10;i++) cout<<t[i]<<" ";
```

- Exécution

```
1 1 1 1 1 2 2 2 2 2
```

Principe (1/2)

- Avant agrandissement

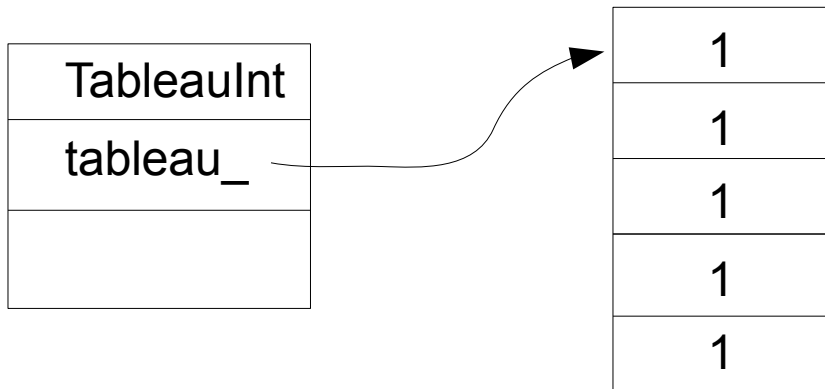


Tableau dynamique

- Après agrandissement

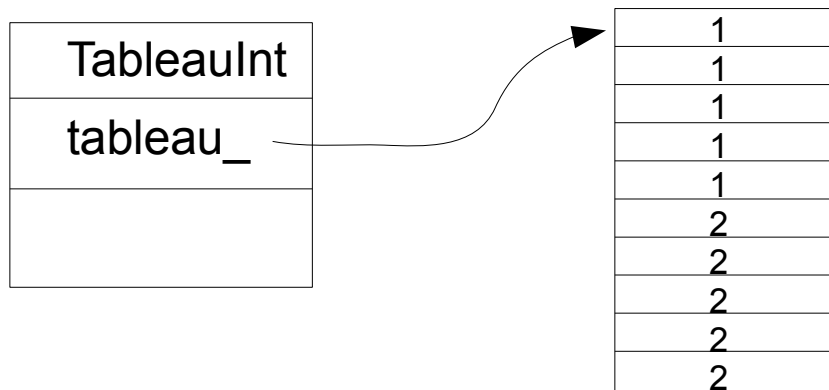
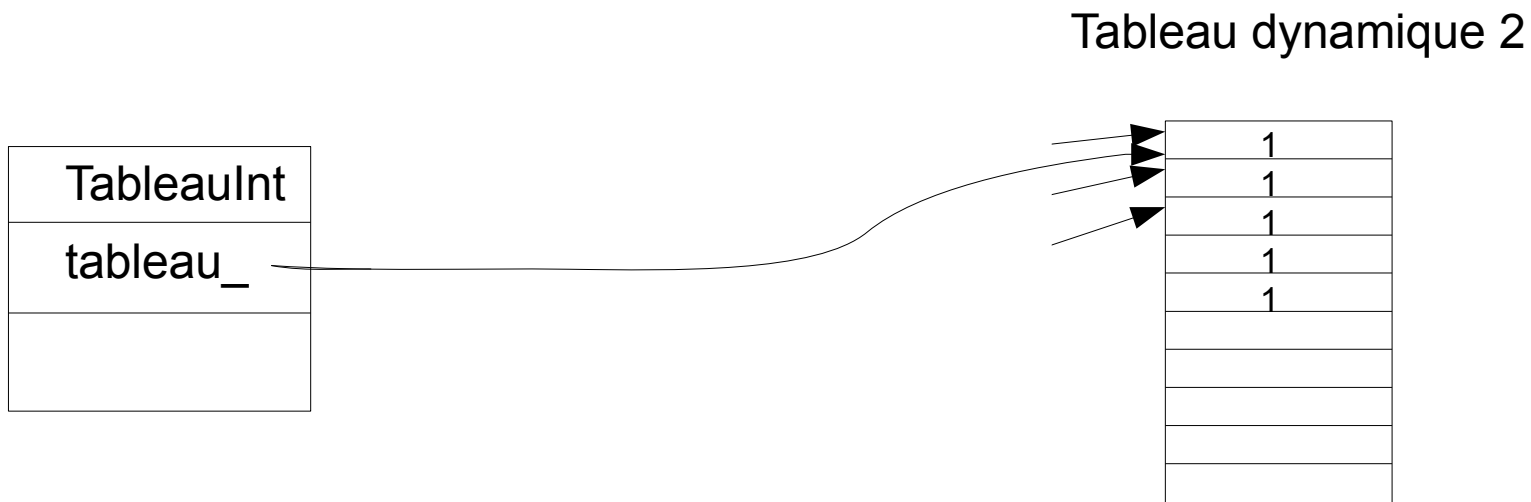


Tableau dynamique

Principe (2/2)

- Remplacer le tableau dynamique
- Recopier les valeurs
- Supprimer le premier tableau
- Faire pointer sur le nouveau



Mise en oeuvre

```
void TableauInt::setTaille(int taille)
{
    int *p,i,min;
    if (taille>=0)
    {
        p=new int[taille];                // Nouveau tableau
        if (taille_<taille) min=taille_;
        else min=taille;
        for (i=0;i<min;i++) p[i]=tableau_[i]; // Recopie
        delete [] tableau_;                // Destruction de l'ancien
        tableau_=p;                          // Pointe sur le nouveau
        taille_=taille;
    }
}
```

Ajout et retrait à la fin

- Ce que l'on veut faire : gestion comme une pile

```
TableauInt t(5);
```

```
int i;
```

```
for (i=0;i<5;i++) t[i]=1;
```

```
t.ajouterDernier(2);
```

```
t.ajouterDernier(3);
```

```
for (i=0;i<7;i++) cout<<t.supprimerDernier();
```

- Exécution

```
3 2 1 1 1 1 1
```

Mise en oeuvre (1/2)

```
void TableauInt::ajouterDernier(int val)
{
    setTaille(taille_+1);
    tableau_[taille_-1]=val;
}
```

Mise en oeuvre (2/2)

```
int TableauInt::supprimerDernier()
{
    int val=0;
    if (taille_>0)
    {
        val=tableau_[taille_-1];
        setTaille(taille_-1);
    }
    return val;
}
```

Copie de tableau

- Ce que l'on veut faire
- Principe
- Mise en oeuvre

Ce que l'on veut faire

- Copie de tableaux

```
TableauInt t1(5);
```

```
int i;
```

```
for (i=0;i<5;i++) t1[i]=1;
```

```
TableauInt t2;
```

```
t2=t1;
```

```
for (i=0;i<5;i++) cout<<t2[i]<<" ";
```

- Exécution

```
1 1 1 1 1
```

Principe (1/3)

- Ajouter une méthode

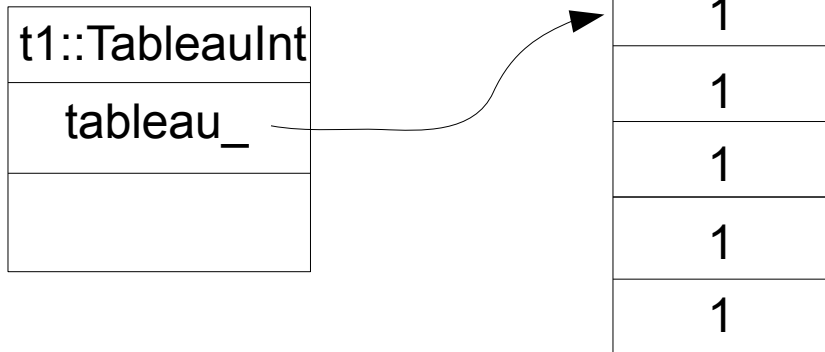
`void operator = (const TableauInt &t)`

`t2=t1    ⇔    t2.operator = (t1);`

Principe (2/3)

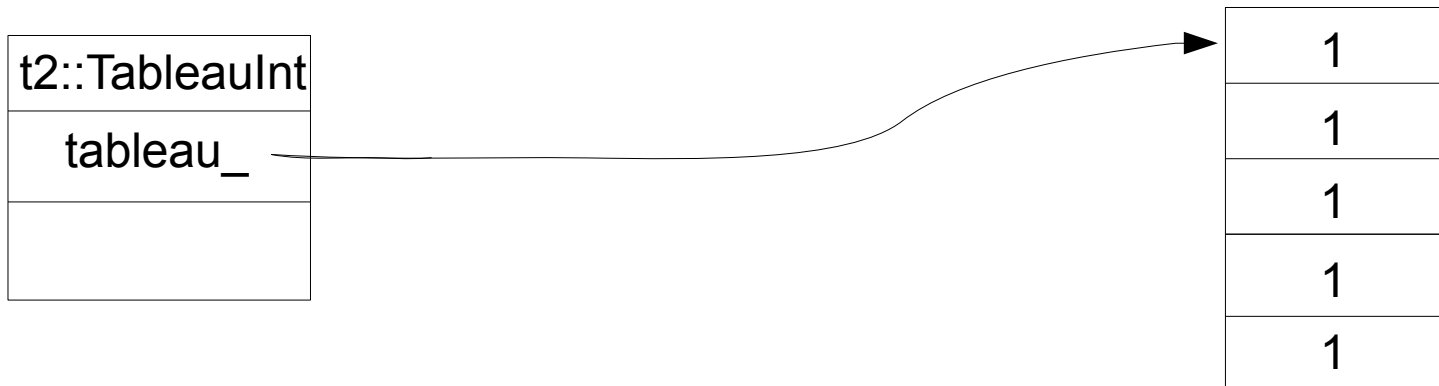
Tableau dynamique de t1

t1



Nouveau tableau dynamique

t2



Principe (3/3)

- Supprimer l'ancien tableau
- Créer un nouveau tableau dynamique pour t2
- Recopier les valeurs du tableau de t1

Mise en oeuvre

```
void TableauInt::operator=(const TableauInt &t)
{
    int i;
    if (taille_!=t.taille_)
    {
        delete [] tableau_;
        tableau_=new int[t.taille_];
        taille_=t.taille_;
    }
    for (i=0;i<taille_;i++) tableau_[i]=t.tableau_[i];
}
```

Constructeur de copie

- Passage d'objet par valeur
- Propriétés du constructeur de copie
- Mise en œuvre

Passage d'objet par valeur

- Exemple

```
void afficher(TableauInt t)
{
    for (int i=0;i<t.getTaille();i++)
        cout<<t[i]<<" ";
    cout<<endl;
}
```

```
void main()
{
    TableauInt tab(5);
    for(int i=0;i<5;i++) tab[i]=i;
    afficher(tab);
}
```

Lors de l'appel à Afficher :

- Création d'une copie de *tab*
- Transmission de la copie *t* à la fonction Afficher

⇒ Appel d'un constructeur spécial : constructeur de copie

Propriétés du constructeur de copie

- Possède un paramètre qui est un objet de la classe

`TableauInt(const TableauInt &t)`

- Est appelé lorsqu'il y a une copie d'objet
 - Automatiquement lors d'un passage de paramètre par valeur
 - Explicitement pour initialiser à partir d'un objet existant

```
TableauInt t1(5);
```

```
for(int i=0;i<5;i++) t1[i]=i
```

```
TableauInt t2(t1);           // Appel du constructeur de copie23
```

Mise en œuvre

```
TableauInt::TableauInt(const TableauInt& t)
{
    int i;
    taille_=t.taille_;
    tableau_=new int[taille_];
    for (i=0;i<taille_;i++) tableau_[i]=t.tableau_[i];
}
```

Remarque importante sur = et le constructeur de copie

- S'ils n'existent pas dans une classe
 - Ces opérateurs sont automatiquement définis par le compilateur
 - Pour que les passage par valeur et copies d'objet soient possible
 - Leur comportement est une copie (simple) des champs :

```
taille_=t.taille_;  
tableau_=t.tableau_;
```
 - Cela ne convient pas pour la classe TableauInt
 - Cela convient pour une classe sans partie dynamique

Déclaration finale de la classe

```
class TableauInt
{
    int * tableau_;
    int taille_;
public :
    TableauInt(int taille=0);
    TableauInt(const TableauInt& t);
    ~TableauInt();
    int getTaille() const;
    int getElement(int i) const;
    void setElement(int i,int val);
    int & operator[ ](int i);
    void setTaille(int taille);
    TableauInt& operator=(const TableauInt &t);
    void ajouterDernier(int val);
    int supprimerDernier();
    void afficher() const;
};
```

Exercice 3

Dans cet exercice, on utilisera la classe *TableauInt* du cours pour manipuler des tableaux d'entiers.

- a) Ecrire une fonction *concatenerTableaux* ayant 2 tableaux *t1* et *t2* en paramètres et qui agrandit le tableau *t1* pour lui ajouter à la fin les éléments de *t2*.
- b) Ecrire un programme permettant de tester la fonction précédente.